Deployment guide

Real-Time Cloud-Based Student Performance Analytics Dashboard on AWS CloudFormation

Everything in the project is created by one CloudFormation stack: storage, database, authentication, API, real-time channel, processing functions, permissions and monitoring. Only two things are uploaded afterwards, because CloudFormation cannot upload files: the sample CSV and the dashboard pages. The deploy script does both for you.

1. What the stack creates

Synopsis component	AWS resource in the template
Web app and dashboard	S3 bucket with static website hosting (your dashboard design)
Authentication and roles	Cognito user pool, app client, groups teacher / hod / admin, first admin user
API service	API Gateway HTTP API with a Cognito JWT authorizer and one Lambda function
Cloud database	DynamoDB tables: students, records (stream on), summary, connections
Processing service	Lambda processor (loads uploaded CSV) and Lambda stream (updates summaries on every write)
Analytics engine	Running totals per class and section in the summary table, updated by the stream Lambda
Real-time update service	API Gateway WebSocket API and Lambda ws; the stream Lambda pushes new rows to open dashboards
Cloud storage	Private, encrypted S3 data bucket; CSV files go in the raw/ folder
Monitoring and logging	CloudWatch log groups with retention, plus alarms for Lambda errors and API 5xx

2. Before you start

• An AWS account, with a billing alert set in AWS Budgets. The design stays inside the free tier for a demo of about 100 students.

• The AWS CLI version 2, signed in with `aws configure`. Pick one region and use it for everything. Check with the first command below.

• Python 3.9 or newer, used by the build script. On Windows use Git Bash or AWS CloudShell to run the shell script.

• Unzip `cloud-student-performance-analytics.zip` and open a terminal inside the folder.

```
aws sts get-caller-identity          # shows your account, so you know the login works
aws configure get region               # shows the region the stack will use
```

3. Method A: deploy with one command (recommended)

Step 1. Choose the admin sign-in

The admin email is the login for the dashboard. The password needs 8 or more characters with upper case, lower case and a number. Passwords are typed only into your own terminal; never put them in a file or in GitHub.

```
export ADMIN_EMAIL='you@example.com'
export ADMIN_PASSWORD='ChangeMe123'
```

Step 2. Run the deploy script

The script builds the template, validates it, creates the stack (about 3 to 5 minutes), sets the admin password, uploads the sample CSV, writes the dashboard's config.js from the stack outputs and publishes the dashboard.

```
./deploy.sh
```

Step 3. Open the dashboard

The last line printed is the dashboard address. Open it, sign in with the admin email and password, and the KPI ribbon and charts appear. The sample data has 90 students, 6 subjects and 6 assessments (3,240 records).

4. Method B: deploy from the AWS Console

Step 1. Build the template

Run this once on your computer. It places the Lambda code inside cloudformation/template.yaml.

```
python3 build_template.py
```

Step 2. Create the stack

Console > CloudFormation > Create stack > With new resources > Upload a template file > choose cloudformation/template.yaml. Stack name: **student-analytics-dev**. Enter your AdminEmail; keep the other parameters. Add tags Project, Environment, Owner and Purpose. On the last page tick the box that acknowledges IAM resources will be created, then submit.

Step 3. Wait for CREATE_COMPLETE

Open the Events tab to watch progress. When the status is CREATE_COMPLETE, open the Outputs tab and keep it open: you need HttpApiUrl, WebSocketUrl, UserPoolId, UserPoolClientId, DataBucketName and DashboardBucketName.

Step 4. Set the admin password

Cognito created the admin user without a password. Run this in your terminal or in AWS CloudShell (replace the two values).

```
aws cognito-idp admin-set-user-password --user-pool-id <UserPoolId> \
  --username you@example.com --password 'ChangeMe123' --permanent
```

Step 5. Upload the sample data

Console > S3 > the data bucket > create a folder named raw > upload data/sample_student_data.csv into it. The upload starts the processor Lambda automatically.

Step 6. Configure and upload the dashboard

Edit dashboard/config.js so it holds your stack outputs, then upload all files in the dashboard folder to the dashboard bucket (DashboardBucketName). Open the DashboardUrl output.

```
window.APP_CONFIG={region:"<your region>",apiUrl:"<HttpApiUrl>",
  wsUrl:"<WebSocketUrl>",clientId:"<UserPoolClientId>"};
```

5. Verify that everything works

Check	Where to look	Expected result
Stack	CloudFormation > Stack > Resources / Outputs	CREATE_COMPLETE, all resources listed, outputs filled
Data upload	S3 data bucket > raw/	sample_student_data.csv is present
Processor Lambda	CloudWatch > Log groups > .../processor	A line saying Loaded 3240 records
Database	DynamoDB > Tables > records > Explore items	3,240 records; the students table has 90 items
Summaries	DynamoDB > summary table	An ALL item plus one item per class and section
Stream Lambda	CloudWatch > Log groups > .../stream	Invocations with no errors
Dashboard	Dashboard URL	Sign in, then 8 KPI cards, 6 charts and the student table

Test the API and the live update. Open the dashboard, then run these commands in a terminal. The saved mark appears in the Live data feed within a few seconds without reloading the page.

```
TOKEN=$(aws cognito-idp initiate-auth --client-id <UserPoolClientId> \
  --auth-flow USER_PASSWORD_AUTH \
  --auth-parameters USERNAME=you@example.com,PASSWORD='ChangeMe123' \
  --query AuthenticationResult.IdToken --output text)

curl -H "Authorization: $TOKEN" <HttpApiUrl>/summary

curl -X POST <HttpApiUrl>/records -H "Authorization: $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"studentId":"S001","subject":"Cloud Computing","period":"2026-09",
       "sem":"Semester 4","date":"2026-09-21","exam":"Internal assessment",
       "marks":91,"attendance":94}'
```

Marks or attendance outside 0 to 100, or an unknown student, return a 400 error. Saving the same student, subject and assessment again replaces the old row, so duplicates cannot occur.

6. Add teachers and other users

Roles are Cognito groups. Teachers and admins can save marks; head of department and admin accounts can view everything. Create a teacher like this, then ask them to sign in with the password you set.

```
aws cognito-idp admin-create-user --user-pool-id <UserPoolId> --username teacher@example.com \
  --user-attributes Name=email,Value=teacher@example.com Name=email_verified,Value=true \
  --message-action SUPPRESS
aws cognito-idp admin-set-user-password --user-pool-id <UserPoolId> \
  --username teacher@example.com --password 'Teacher123' --permanent
aws cognito-idp admin-add-user-to-group --user-pool-id <UserPoolId> \
  --username teacher@example.com --group-name teacher
```

7. Screenshots to capture for your report

• CloudFormation: stack CREATE_COMPLETE, Resources tab, Outputs tab, Events tab.

• S3: both buckets, and raw/sample_student_data.csv in the data bucket.

• DynamoDB: the four tables, and a few records in the records table.

• Cognito: user pool, the three groups and the admin user.

• Lambda: the four functions, and CloudWatch log lines from processor and stream.

• API Gateway: the HTTP API routes with the JWT authorizer, the WebSocket API, and a successful /summary response.

• Dashboard: sign-in page, KPI ribbon, charts, student table, and the Live data feed after a POST.

• Hide your account ID if you wish. Never capture passwords, tokens or access keys.

8. Update, remove and costs

Task	How
Change the template or Lambda code	Edit lambda/*.py or cloudformation/template.base.yaml, run ./deploy.sh again. CloudFormation updates only what changed.
Change the dashboard	Edit files in dashboard/, then run: aws s3 sync dashboard s3://
Load more data	Upload another CSV with the same columns into raw/ in the data bucket.
Delete everything	./deploy.sh destroy. It empties both buckets first, because CloudFormation cannot delete a bucket that still has files, then deletes the stack.
Costs	DynamoDB on-demand, Lambda, API Gateway and Cognito are pay-per-use and cost almost nothing at demo scale. Delete the stack when you finish.

9. Troubleshooting

Problem	Likely cause and fix
Stack ends in ROLLBACK_COMPLETE	Open Events and read the first FAILED line. Delete the failed stack, fix the cause, deploy again.
Bucket name already exists	Names include your account ID, so this means an old stack is still present. Delete it first, or change ProjectName.
Sign-in says incorrect username or password	The admin password was not set. Run the admin-set-user-password command in step 4 of Method B.
Dashboard shows an error after sign-in	config.js still has empty values. Re-run ./deploy.sh or upload the edited file to the dashboard bucket.
Charts are empty	The CSV has not been loaded. Check raw/ in the data bucket and the processor log group.
API returns 401	Use the ID token, not the access token, and get a new one: tokens last one hour.
No live updates	Open the browser console for WebSocket errors. Sign in again to get a fresh token and check the stream Lambda log group.
Cannot delete the stack	A bucket still has files. Use ./deploy.sh destroy, or empty both buckets in the console first.